WISEDRAM: A Reliable Bitwise In-DRAM Accelerator

Mohammad Arman Soleimani[1], Nezam Rohbani[2,3], Adrian Cristal Kestelman[2], Osman Unsal[2], Hamid Sarbazi-Azad[1,3]
[1]Department of Computer Engineering, Sharif University of Technology, Iran
[2]Barcelona Supercomputing Center, Spain
[3]Institute for Research in Fundamental Sciences (IPM), Iran
{arman.soleimani, azad}@sharif.edu {nrohbani, osman.unsal, adrian.cristal}@bsc.es

2025 62nd ACM/IEEE Design Automation Conference (DAC) | 979-8-3315-0304-8/25/$31.00 ©2025 IEEE | DOI: 10.1109/DAC63849.2025.11133397

***Abstract*—Processing-in-Memory (PIM) aims to address the costly data movement between processing elements and memory subsystem, by computing simple operations inside DRAM in parallel. The large capacity, wide activation size during cell access, and the maturity of DRAM technology, make this technology a great choice for PIM techniques. Nonetheless, vulnerability to process variation and noises, internal leakage of the cells, and high latency in cell access, limit the utilization of processing in DRAMs for real-world applications. This work proposes a fast PIM technique, called WISEDRAM, which leverages one row of special cells, called X-cells, to enable in-DRAM bulk-bitwise operations. Unlike previous approaches, WISEDRAM retains the conventional DRAM cell access procedure, thereby ensuring the reliability of cell access for reads and writes at a level equivalent to that of conventional DRAMs. Compared with the state-of-the-art, WISEDRAM exhibits 22% reduction in average bitwise computation latency and a 71% improvement in XOR operation execution speed, while imposing an area overhead of 1.6%.**

***Index Terms*—DRAM, Processing-In-Memory, Bitwise Operation, Memory Wall**

I. Introduction

Transferring data between memory and processing elements, such as the CPU or GPU is energy-intensive and suffers from high latency [12], [22]. Furthermore, due to limited datapath width, i.e. 64 bit in DDR5, memory access is serialized, which exacerbates memory access latency. This problem, which results in a performance bottleneck, is so severe that it has been termed the ”memory wall”. Memory-intensive applications, such as neural network processing and graph processing, are particularly vulnerable to the memory wall effect, and unpredictable memory accesses further worsen the problem by reducing the effectiveness of cache hierarchy [6], [23].

To reduce the amount of data transmission between the memory subsystem and processing cores, various techniques have been proposed, including Near Memory Processing (NMP) and Processing In Memory (PIM) [21], [24].

In NMP, processing logic, such as small processing units, is placed near the memory. This reduces the latency between the two units, and relaxes the memory wall but does not entirely eliminate it. There is data movement between the processing logic and memory, thus the problem still fundamentally exists. Moreover, NMP schemes cannot fully harness the parallelism that exists in memory arrays and offer less throughput potential than processing directly on the memory arrays [24].

On the other hand, PIM involves enabling processing inside memory arrays and performing operations on stored data inside the memory. This can be achieved by adding logic to each column of memory, or by exploiting analog properties and available memory elements. Unlike NMP, PIM is able to fully exploit the huge amounts of parallelism available inside memory arrays, and completely eliminate the memory wall, as there is no processor-memory data movement.

Various PIM techniques have been presented for different types of memory technologies, including Non-Volatile Memories (NVMs) [26], ReRAMs [4], SRAMs [2], and DRAMs [28]. The 1-transistor, 1-capacitor DRAM (1T1C-DRAM) structure has gained significant attention for PIM due to its high capacity and massive internal parallelism, maturity of its technology, and the costly off-chip data movement between main memory and processor. The memory wall is often observed between DRAM and the processor, since on-chip cache has considerably less latency compared to off-chip DRAM, and the vast majority of computer systems rely on DRAM as main memory.

However, a practical implementation of PIM in conventional DRAM devices faces unique challenges. The highly compact nature of the 1T1C-DRAM arrays, with bitline pitches of approximately 3 F to 4 F (F = Feature size) [14], makes it extremely difficult to accommodate additional PIM-related circuits or modifications, as these must be compatible with the DRAM fabrication process. This poses a significant challenge, as DRAM chips are fabricated with less metal layers compared with typical CMOS chips which implement processing logic [14], [32]. As a result, integrating logic into DRAM chips is often complex and challenging.

This work presents WISEDRAM, a novel technique for in-DRAM processing, which aims to improve upon the state-of-the-art. The proposed method adds one row of a special dual-contact memory cell, called X-cell, to the DRAM array. This row of cells is capable of performing any two-operand bitwise operation in three DRAM cycles, which is the least amount of cycles theoretically possible - two operand reads and one result write - and significantly faster than current methods.

WISEDRAM's fast and robust computations are shown to greatly improve upon the state-of-the-art in terms of robustness, and our simulations show that this is a strong concern with the state-of-the-art. The approach of enabling computation through a row of specialized cells means that other parts of the memory structure, such as the decoders, peripherals, and amplifiers, are not modified. Thus, WISEDRAM is less intrusive in terms of impact on chip development.

To make the case for WISEDRAM's design and the X-Cell, we conduct a qualitative and quantitative comparison against four state-of-the-art PIM techniques, Ambit [31], ROC [33], ELP2IM [34], and PIPF-DRAM [28], and our technique outperforms these techniques in terms of performance, particularly in performing X(N)OR operation, with a speed improvement of up to 71%. On average, WISEDRAM is faster than the

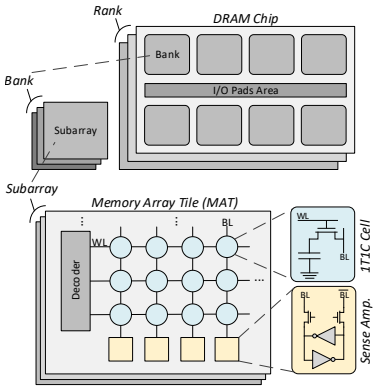

Fig. 1: DRAM chip general structure.

fastest technique presented in the literature to perform bulk-bitwise operations by 22%. WISEDRAM computes any two-operand bitwise operation in three DRAM cycles, which is the least possible number of cycles. Additionally, WISEDRAM is more robust against process variation, achieving a robustness improvement of 77% when compared with the most robust processing in DRAM technique in the literature. The proposed technique incurs a negligible area overhead of only 1.6%.

II. Preliminaries

A. DRAM Fundamentals

1) DRAM Structure: The most widely used DRAM cell structure is 1T-1C (One Transistor-One Capacitor) configuration, where each memory cell consists of a capacitor as the storage element and an access transistor [14].

In the typical structure of DRAM memories, the memory cells are arranged in a two-dimensional layout within a Memory Array Tile (MAT), with dimensions such as 512×512 or 1024×1024 memory cells. Access to these memory cells is facilitated through horizontal and vertical wires known as Wordlines (WLs) and Bitlines (BLs), respectively [5], [19]. Local wordline drivers are activated by a row clock/address spine, which is in turn controlled by global wordlines. A differential *sense amplifier* (SA) at each bitline pair detects the tiny voltage perturbations during cell access. Notably, adjacent MATs in the DRAM chip share SAs as well as wordline drivers.

An overview of DRAM's structure hierarchy can be seen in Figure 1. Each DRAM chip comprises multiple banks, which in turn comprise multiple sub-arrays. Finally, each sub-array is made up of multiple MATs, described earlier.

2) DRAM Operation: Three main operations in DRAM memories - Read, Write, and Refresh - are closely related. In Read and Write, data is transferred in opposite directions, whereas in a Refresh operation - required to restore the leaked charge in DRAM cells - only cell access occurs without data transfer [14]. Prior to accessing a cell, all the bitlines are precharged to half-VDD (*VDD/2*) and then left floating. This phase is known as the *precharge* phase in DRAM cells access. Then, to access a cell during the *activation* phase, the corresponding wordline is activated, causing all the memory cells, each consisting of a tiny capacitor, to connect to the corresponding bitlines. This results in a voltage perturbation (V_S) of about 100 mV on the floated bitline by charge sharing.

Due to the environmental and internal noise, differential sensing is essential to reliably read the V_S value. Thus, bitlines are arranged in pairs connected to differential SAs. One of the bitlines in each pair is left floating, with its voltage expected to be at *VDD/2*, and this voltage is used to detect positive or negative voltage perturbations on the other bitline in the pair. Since the stored charge in the accessed cell is either *0* or *VDD*, the voltage of the bitline connected to the cell will be either lower or higher than the reference (floating) bitline. Using this scheme, the SA cancels out potential noise on the bitline pair and reliably detects the value on the accessed cell.

The structure of a differential SA, as illustrated in Figure 1, typically consists of cross-coupled inverters with strong positive feedback, allowing voltage comparison. The input/output with a higher voltage is rapidly pulled to *VDD*, while the other one is pulled to *GND*. During the precharge phase, the SA power is gated from the VDD and GND power rails, resulting in the internal nodes of the SA being precharged to *VDD/2* alongside the bitlines.

To prevent voltage drop while charging or discharging bitcell capacitors to/from *VDD* using the NMOS access transistor, the controlling voltage of wordlines should be set higher than *VDD*+V_{Th}, where V_{Th} represents the threshold voltage of the NMOS access transistor. This higher voltage level is known as the High-Level VDD or V_{DH}. In the case of DDR5, this voltage is provided by the 1.8 V input pins, also known as *VPP* [1].

When a cell is connected to a bitline, the stored data/charge in the cell is destroyed due to charge sharing operation. To restore the cell charge, the corresponding bitline, which is connected to the accessed cell, is charged/discharged to the same charge level as the cell (before the access) during the stabilization phase of the SA [7], [14].

Once the SA is stabilized to around 3/4 of *VDD*, data can be accessed by activating the Column Select Line [7], [27]. The data is differentially transferred to the global sense amplifier to improve the speed and reliability of data transfer, and the data becomes available at the output.

B. Related Work

PIM techniques that directly utilize DRAM cells for processing can be divided into two categories: Analog Processing In-DRAM and Semi-Digital Processing In-DRAM. Other techniques, such as Logic-Placement In-DRAM [5], [15], [17], [35] and In-DRAM Look-Up Tables, are not considered in this study [8], [9], [36], [37].

Analog Processing In-DRAM: This category employs charge sharing among multiple activated DRAM cells for bitwise processing [3], [31]. Using charge-sharing, an analog majority function is performed, where the charge of the majority of cells determines the value read on the SA. In [31], concurrent triple-row activation (TRA) is presented to enable bitwise AND/OR operations using the majority function. The NOT function is achieved using additional dual-contact cell (DCC) rows [31]. In [3], [29], a bit-serial adder utilizing a 5-bit analog majority function is implemented, and operands and operation results are read and stored in a column-based organization to avoid shift operations between bitlines. However, this requires a costly transpose operation to write/read data to/from DRAM.

The main and most critical limitation of analog PIM is its susceptibility to process variation, ambient and internal noises, and cell charge leakage [25]. This is because the perturbation signal (Vs) during multiple-cell activations, is smaller than a single cell in the worst-case scenario, when the numbers of '0's

and '1's stored in the activated cells are close to each other. Another challenge is that the initial operand data are destroyed by performing the PIM operation, necessitating the use of multiple bulk copies [17]. Furthermore, the operation result is stored in multiple cells, resulting in extra energy dissipation.

Semi-Digital Processing In-DRAM: Semi-digital PIM approaches activate only a single wordline at a time. In [10], AND and OR operations are achieved using off-the-shelf DRAMs by sequentially activating operand rows at a high frequency, simulating simultaneous activation of multiple rows. However, this technique does not support NOT operation, rendering complete logic realization infeasible. Moreover, controlling the timing of wordline voltage and the high-frequency voltage swing on the wordlines is challenging, causing increased leakage current and thus compromising reliability goals. [14].

To overcome these, a technique presented in a [34] eliminates extra bulk copy and error-prone charge sharing between multiple cells by utilizing additional voltage control states in the precharge circuit and the SA. This technique does not require reserved cell rows for operation, but rather drives the sense amplifier pull-down circuit with a voltage of half-VDD instead of GND during the operation. However, this significantly reduces the SA drive current and stability by approaching the threshold voltage of the transistors to VDD, which affects overall reliability. Additionally, the NOT operation is performed using extra DCC rows.

Another work [33] leverages the characteristics of a diode to implement logic operations. In this technique, two extra DRAM rows are utilized, where the cells in each column are connected together with a diode to perform OR and AND operations. This approach is faster than previous techniques, the charge of the cell containing the operation result is degraded by the threshold voltage drop on the diode, which significantly reduces its reliability. One of the advantages of semi-digital PIM over analog PIM is avoiding error-prone charge sharing between cells. However, as shown in this study, these methods still severely hurt reliability. Moreover, they require more modifications to the DRAM structure for the desired functionality.

The most relevant previous methods include Ambit [31], ROC [33], ELP2IM [34], PIPF-DRAM [28], and Lacc [8].

Ambit employs charge sharing among multiple cells containing operands by connecting them simultaneously, and utilizes the final developed voltage (V_S) on the bitline to perform a majority function. The majority function is then used to perform more complex bitwise operations. In Ambit, additional DRAM cell rows are considered and activated simultaneously, with operands copied to these rows before the operation begins.

The key idea of ROC is to leverage the diode-like behavior of an NMOS transistor with the gate connected to its sources, allowing for cell charging while preventing discharge in the opposite direction. This technique employs basic bitwise operations such as NOT, AND, and OR to perform more complex operations like XOR.

ELP2IM and PIPF-DRAM, on the other hand, utilize the remaining charge on the bitlines from the previous cell access to perform bitwise operations. PIPF-DRAM is based on a special DRAM structure called PF-DRAM [27], which eliminates the precharge phase and modifies the controlling signals to enable bitwise operations. Lacc [8] is another processing-in-DRAM technique aiming at CNN acceleration which performs X(N)OR operations as well. However, this technique is not considered in our evaluations due to its bitwise computing abilities being limited to X(N)OR.

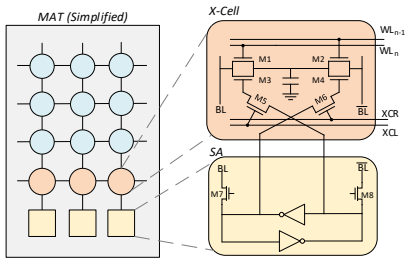

Fig. 2: Structure of WISEDRAM. Precharge circuitry and SA boosters are omitted for readability.

III. Proposed PIM Technique

In this section we describe the proposed technique for processing in memory, called WISEDRAM (a Reliable Bitwise In-DRAM Accelerator). Then we discuss different bitwise operations that WISEDRAM can perform.

A. WISEDRAM Structure

WISEDRAM integrates a row of special cells, called X-Cells, into conventional DRAM for bitwise operations, as shown in Figure 2. X-Cells resemble dual-contact cells [18], [31], with four added transistors (*M3* to *M6*) that allow access from either bitline based on the previous SA read.

By activating the XCL (X-Cell Connect Left) and XCR (X-Cell Connect Right) control signals, the X-Cell induces a voltage perturbation on either the left or right bitline, depending on whether the previous value, from the previous cell access, on the SA is '0' or '1', respectively. This ability can be leveraged to perform bitwise operations. While the provided examples focus on operands connected to BLL (Bitline Left) for clarity, it's worth noting that the operands can be on any MAT cell, thanks to the X-Cell's dual-contact nature.

WISEDRAM retains the same structure and memory cell access operations as conventional DRAM. It is important to note that the SA structure was not modified, unlike many of the previously proposed techniques discussed in Section IV. In conventional DRAMs, decoupler transistors (*M7* and *M8*) are controlled by the DCL (Bitline Decoupler) signal to enhance memory access latency and enable the sharing of SAs between adjacent MATs. In the DDR5 architecture, the activation of a row of 64 Kb during each cell access enables the parallel operation of 64 Kb through the use of WISEDRAM. This technique leverages the substantial internal bandwidth of DRAM to achieve significant acceleration of bitwise operations.

B. X(N)OR in WISEDRAM

Due to the significance of X(N)OR as a major bitwise operation, WISEDRAM structure has been optimized for efficient X(N)OR operations. The rationale behind WISEDRAM's design is based on the XOR operation between two operands, *A* and *B* ($A \oplus B$).

In order to compute $A \oplus B$ serially, the value of one operand (here *B*) is observed. If $B = 0$, it can be seen that $A \oplus B = A$. Otherwise, $A \oplus B = \overline{A}$. This view of XOR as a "controlled NOT" is the fundamental operation of WISEDRAM.

The computation steps for XOR in WISEDRAM are illustrated in Figure 3, and Figure 4 shows the respective waveforms, extracted from simulations explained in Section IV.

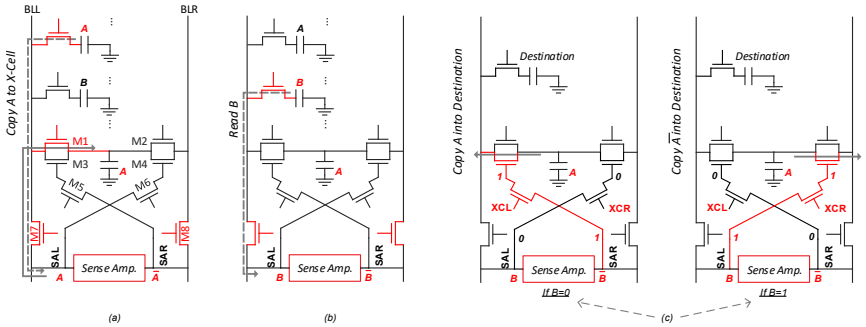

Fig. 3: XOR Computation in WISEDRAM: (a) Copying *A* to X-Cell, (b) Reading *B*, (c) Accessing X-Cell via XCL and XCR to copy X-Cell to Destination. For XNOR operation, *A* must be copied from *M2* in the first step, thus storing the complement of *A* in the X-Cell.

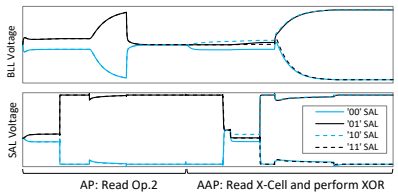

Fig. 4: Waveform of BLL and SAL in WISEDRAM while computing XOR.

To begin, one of the operands (in this case, *A*) is copied to the X-Cell using a row copy as introduced in RowClone [30]. This is achieved through an overlapped Activate-Activate-Precharge (AAP) primitive [31], which can be observed in Figure 3 (a). The X-Cell is a dual-contact cell that features access transistors connected to either bitlines in each pair. With this structure, operands can be stored on any cell in the memory array.

In the second step, as shown in Figure 3 (b), the second operand (*B*) is read, causing its value to appear on the SA nodes. The left SA node, SAL, stabilizes at the value of *B*, while the right node, SAR, takes on the complement value ($\overline{B}$). Once *B* has been restored, the bitlines are precharged and equalized to *VDD/2*, the same as conventional DRAM cell access. This step is an Activate-Precharge (AP) primitive.

In the final step of X(N)OR operation, the control signals XCR and XCL activate *M5* and *M6*, as shown in Figure 3 (c). If $B = 0$, the voltage on SAR activates *M3*, while *M4* remains inactive. In this case, the X-Cell is accessed from the BLL, and the voltage perturbation of *A* appears on BLL. Alternatively, if $B = 1$, the value of SAL activates *M4*, and *M3* remains inactive. In this case, the X-Cell is accessed from the BLR (Bitline Right), and the voltage perturbation of *A* appears on BLR. After accessing the X-Cell, the SA is equalized and connected to the bitlines for reading the value, through *M7* and *M8*. The read value is A if $B = 0$, and $\overline{A}$ if $B = 1$. The wordline for the target cell is also activated during this step, and this activation is overlapped [30], making the third step an AAP primitive. This step is similar to a row copy, but instead of activating the source (X-Cell) using a wordline, it is activated using XCR and XCL signals. As a result, the value $A \oplus B$ is written back to the selected destination operational cell.

The computation of XNOR is similar to XOR. Eq. 1 serves as the basis for XNOR computation in WISEDRAM. Since the X-Cell is dual-contact cell, instead of copying A in the first step (see Figure 3 (a)), $\overline{A}$ can be copied in X-Cell. By continuing the steps as outlined in Figure 3, the resulting value stored in the destination cell will be $\overline{A} \oplus B$, which corresponds to the XNOR of *A* and *B*.

$$\overline{A \oplus B} = \overline{A} \oplus B \tag{1}$$

The procedure for performing the X(N)OR operation indicates that it can be executed in three DRAM AP/AAP primitives (one AP and two AAP). This is almost as fast as the theoretical optimal number of operations required (three AP primitives), since an access to two operands and one destination is inevitable.

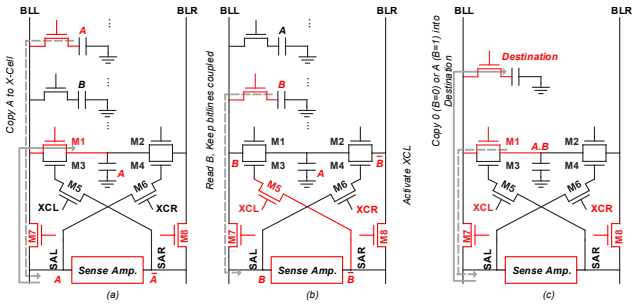

Fig. 5: AND Computation in WISEDRAM: (a) Copying *A* to X-Cell, (b) Reading *B* and activating XCL, (c) Copying X-Cell to Destination. For NAND, the complement of the X-Cell can be copied into the destination cell.

C. (N)AND and (N)OR in WISEDRAM

This subsection explores how WISEDRAM's X-Cell can be leveraged to perform (N)AND and (N)OR operations in three DRAM AP/AAP primitives (one AP and two AAP), which is almost as fast as the theoretical minimum time required to compute these operations (three AP primitives).

1) (N)AND Computation: The computation of (N)AND follows a similar procedure to XOR, but with a distinct application of the X-Cell. The logical operation of AND between two operands, represented as $A.B$, can be formulated using Eq. 2. The result of the operation is '0' if the first operand is '0', irrespective of the second operand. On the other hand, if the first operand is '1', the output is equivalent to the second operand. This principle underlies the computation of AND in WISEDRAM, as depicted in Figure 5.

$$A.B = \begin{cases} 0 & B = 0 \\ A & B = 1 \end{cases} \tag{2}$$

Initially, the first operand (designated as A) is copied into the X-Cell. Next, the second operand (designated as B) is read, while the sense amplifier remains coupled and the bitlines are not pre-charged. At this point, the sense amplifier drives the bitlines BLL and BLR with the values of B and its complement $\overline{B}$, respectively. Simultaneously, the XCL signal is activated. If $B =$'0', then the value on the SAR, which is the complement of B, will be equal to '1'. As a result, *M3* will be activated, and the value on the X-Cell will be overwritten by the value of BLL, which is equivalent to '0' in this case. Conversely, if $B =$'1', the value stored in the X-Cell (A) will remain unchanged and X-Cell contains *A.B*.

The final step is to copy the X-Cell into the destination cell using an AAP primitive. A regular row copy will save

AND output into the desired destination row. The second access transistor (*M2*) of X-Cell can be leveraged to copy the complement of computed value into the destination, which is equal to $\overline{A.B}$, NAND operation.

2) (N)OR Computation: Computing (N)OR is akin to (N)AND, with minor differences. If XCR is connected in the second step instead of XCL, BLR will overwrite the X-Cell with '0' when $B =$'1'. This will result in the computation of $A.\overline{B}$. The first step can also be changed to copy $\overline{A}$ to the X-Cell, using its dual-contact feature. This results in the computation of $\overline{A.\overline{B}}$ which is equal to $\overline{A + B}$, NOR of the operands is stored in the X-Cell. Copying X-Cell or its complement into the destination fulfills the computation of NOR or OR, respectively. Thus, (N)OR can also be computed in three AP/AAP.

D. Copy and NOT in WISEDRAM

Row copy functionality is present in conventional DRAM structures, which has been studied in previous literature such as [30]. This operation is utilized for various computations, as demonstrated in the preceding subsections and other research works [28], [31], [33].

Since the X-Cell is inherently dual-contact, it can also be used to invert operands during a copy operation to perform the NOT operation similar to the method described in [31].

E. General-Purpose Computing

Previous work has made the case for more advanced operations in DRAM arrays [3], [13]. Since DRAM bitlines are inherently isolated from eachother, these techniques use bit-serial operations, wherein operands are stored vertically in one bitline. By doing so, a bitwise in-memory approach will be able to compute more advanced operations such as addition and multiplication. This work serves as a drop-in replacement for the circuits that are used in [3], [13]; therefore, the same approach can be leveraged to perform non-bitwise computation on WISEDRAM and the circuit-level improvements will propagate to upper layers.

IV. Experimental Evaluations

In this section, we conduct a comparative analysis between WISEDRAM and the state-of-the-art in-DRAM processing techniques, including Ambit [31], ROC [33], ELP2IM [34], and PIPF-DRAM [28].

For circuit-level evaluation of functionality, performance, robustness, and energy consumption, we employ a 16 nm multi-gate technology model in HSPICE, under the same evaluation conditions. Simulation parameters are summarized in Table I.

A. Functionality and Performance

To assess the latency of bitwise operations in comparison with other state-of-the-art techniques, we implemented PIPF-DRAM, Ambit, ROC, and ELP2IM with all of the optimizations presented in their respective works to enhance latency, and

TABLE I: Circuit-level simulation parameters.

Technology	16 nm PTM-MG
VDD/VPP	1.1 V/1.8 V [1]
MAT size	512×512 cell
C_{Cell}/C_{BL}	24 fF/144 fF [7], [27]
Transistors W/L	SA 10/1, decoupling 2/1, access 1/1, precharge 2/1 [11], M1-M4 1/1, M5-M6 2/1 (Figure 2)
Process Variation	Gaussian distribution: σ = 2.5%, 5%, 7.5%, 10%, 12.5%, and 15% m = nominal value for V_{th}, capacitances, and resistances

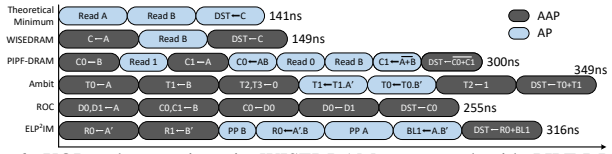

Fig. 6: XOR sub-operations in WISEDRAM, compared with PIPF-DRAM, Ambit, ROC and ELP2IM and the theoretical minimum. In ELP2IM, PP indicates pseudo-precharge.

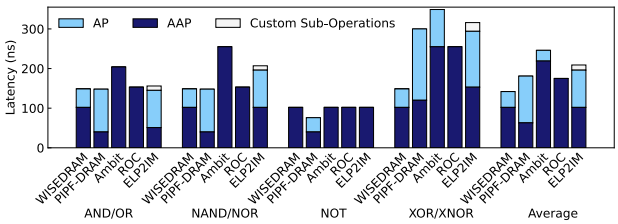

Fig. 7: Latency of operations in WISEDRAM, compared with PIPF-DRAM, Ambit, ROC, and ELP2IM.

then compared their latency with that of WISEDRAM. Figure 6 shows the breakdown of sub-operations to perform XOR in these PIM techniques, alongside the minimum latency that can be theoretically achieved in a PIM technique where the result can be stored in an arbitrary row. Since accessing two operands is inevitable and they must both be restored and maintained, two separate AP primitives are necessary, and another is required to store the result in an arbitrary location, which adds up to three AP operations for a complete XOR computation. WISEDRAM is the closest to this minimum, and requires two AAP primitives and one AP primitive to compute an XOR, which takes 149ns and is only 8ns slower than the absolute minimum.

Figure 7 compares the latency of all bitwise operations. As shown in the figure, WISEDRAM outperforms PIPF-DRAM, Ambit, ROC, and ELP2IM by 101%, 134%, 71%, and 112% for executing X(N)OR, respectively. The AAP operation performs a copy operation between rows, while the AP operation performs cell access. ELP2IM uses some custom sub-operations, other than activation and precharge, to control its special sense amplifier structure to precharge bitlines in an imbalanced manner, which adds extra delay to execute bitwise operations. On average, WISEDRAM exhibits lower latencies compared to PIPF-DRAM, Ambit, ROC, and ELP2IM in executing bitwise operations, with reductions of 27%, 72%, 22%, and 46%, respectively.

B. Energy Consumption

In the execution of operations within the memory structure, the primary contributor to energy consumption is the charge/discharge of high-parasitic capacitance bitlines [28]. This energy dissipation occurs in the sense amplifier pull-up transistors and bitlines precharge circuitry, as illustrated in Figure 1. Another significant energy-intensive operation is the activation of wordlines, which results in energy dissipation within the local address decoders and drivers. WISEDRAM and ROC can perform memory cell access using the same technique as the PF-DRAM structure during PIM operation, similar to PIPF-DRAM. The PF-DRAM structure eliminates the bitline precharge procedure from the cell access process, resulting in a significant reduction in energy consumption and delay during cell access. We evaluated WISEDRAM and ROC on conventional and PF-DRAM structures for fair assessment.

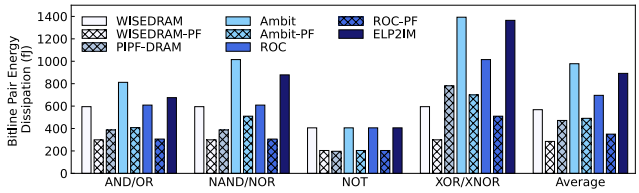

Fig. 8: Energy dissipation on a biline pair using different PIM techniques.

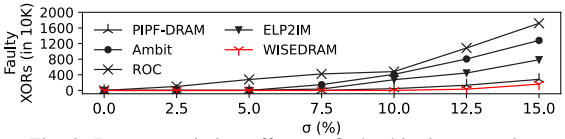

Fig. 9: Process variation effect on faulty bitwise operation.

In conventional DRAM, the AP and AAP operations consume 189 fJ and 203 fJ, respectively. However, in the case of PF-DRAM structure, their energy consumption is reduced to 95 fJ and 102 fJ, respectively. The dissipated energy for bitwise operations on a single bitline using different in-memory techniques is presented in Figure 8. In WISEDRAM, XOR and XNOR operations on a single bit consume 595 fJ, making it 31%, 134%, 70%, and 129% more energy-efficient than PIPF-DRAM, Ambit, ROC, and ELP2IM, respectively. Moreover, by utilizing the PF-DRAM memory access procedure in WISEDRAM, these values are hugely improved to 1.61×, 3.65×, 2.39×, and 3.56×, respectively.

WISEDRAM, equipped with PF-DRAM cell access technique, achieves energy efficiency gains of 29% and 161% over PIPF-DRAM, the most energy-efficient PIM technique among the previously evaluated methods, for the execution of (N)AND/(N)OR, and XOR/XNOR operations, respectively. On average, WISEDRAM (PF) is 99%, 65%, 388%, 72%, 144%, 22% and 212% more power efficient than WISEDRAM, PIPF-DRAM, Ambit, Ambit (PF), ROC, ROC (PF), and ELP2IM, respectively, indicating its superior energy efficiency.

C. Robustness

For an in-memory processing technique to achieve an acceptable yield rate, it must be robust against process variation. To evaluate this, we conducted Monte Carlo simulations using HSPICE, with 10 K rounds of simulations. The operational DRAM cells and sense amplifier transistors' threshold voltages, as well as the parasitic capacitance and resistance of bitlines, were randomly updated in each simulation round using a Gaussian distribution. The standard deviation of the distribution model (σ) was considered to be between 2.5% to 15% of the mean value of each parameter (m).

Figure 9 presents the fault rate versus the standard deviation of variations in parameters for various PIM techniques based on DRAM architecture. Among the evaluated state-of-the-art techniques, ROC shows the highest sensitivity to process variation. Increasing σ from 10% to 15% results in a sharp elevation of fault rate from 482 faults per 10K evaluation rounds to over 1700 faults, respectively. This susceptibility of ROC is attributed to the voltage drop on the diode due to the transistors' threshold voltage, which is ROC's key element for computation.

The next two most susceptible PIM techniques to process variation are Ambit and ELP2IM. Monte Carlo simulations show no faults for these techniques when σ is increased to 5%. However, by increasing σ to 15%, the fault rate increases to 12.8% and 7.84%, respectively.

On the other hand, PIPF-DRAM is the most robust state-of-the-art technique in the literature, with a fault rate of only 284 faults in 10 K simulation rounds at σ=15%. This robustness is attributed to the technique's ability to maintain the perturbation signal during sub-operations. However, PIPF-DRAM employs precisely unbalanced sense amplifiers to perform its operations, which can be easily affected by process variation. WISEDRAM is fault-free even when increasing σ up to 10%. In comparison with ROC, Ambit, ELP2IM, and PIPF-DRAM, the in-memory processing (PIM) technique of WISEDRAM is 9.7×, 6.9×, 3.9×, and 77% more robust against process variation, respectively, at σ=15%. The robustness of WISEDRAM against process variation can be attributed to three main factors. First, WISEDRAM employs a reduced number of sub-operations to perform an operation, thereby reducing the overall sensitivity to process variation-related failures. Second, WISEDRAM uses the same cell access procedure as conventional DRAM, making it less susceptible to process variation. Finally, conventional DRAMs use increased voltages in the sense amplifier [20] to reduce latency. This alleviates the threshold-induced voltage drop in the X-cell's transistors, which caused many errors in ROC.

D. Layout and Area Overhead

The area overhead of WISEDRAM is limited to four extra wordlines per MAT to control X-Cell operation, plus a dual-contact cell and four extra NMOS transistors per bitline pair. As shown in [31], a dual-contact cell occupies around two memory cells. The other controlling transistors, *M3* to *M4* can be fabricated with the smallest size in the technology, the same as cell access transistors, and *M5* and *M6* are slightly larger. The X-Cell can be placed near the sense amplifier to ease layout design, since folded-bitline sense amplifiers are four bitlines wide [16], [32]. The total area overhead of an X-Cell is approximately 8 memory cells per bitline pair. In a 512-cell bitline, this results in an area overhead of roughly 1.6%.

V. Conclusions

Given the maturity, capacity, and large internal bandwidth of DRAM, it is an excellent candidate to serve as a platform for Processing In-Memory (PIM). We present a novel PIM technique called WISEDRAM, which is capable of performing a range of operations including X(N)OR, (N)AND, (N)OR, NOT, and Copy. WISEDRAM can also perform an unlimited number of bitwise operations, providing greater power efficiency and reduced latency. Compared to existing PIM techniques, WISEDRAM executes X(N)OR operations, one of the most complex PIM operations, 71% faster. Additionally, WISEDRAM is 77% more robust against process variation compared to the most robust PIM technique, with only a negligible area overhead of roughly 1.6%.

Acknowledgment

This work is part of the project PID2023-146511NB-I00 funded by the Spanish Ministry of Science, Innovation and Universities MICIU /AEI /10.13039/501100011033 and EU ERDF. This publication is promoted by the Barcelona Zettascale Laboratory, backed by the Ministry for Digital Transformation and of Public Services, within the framework of the Recovery, Transformation, and Resilience Plan – funded by the European Union – NextGenerationEU.

REFERENCES

[1] "Ddr5 sdram," https://www.micron.com/-/media/client/global/documents/products/data-sheet/dram/ddr5/16gb_ddr5_sdram_diereva.pdf, 2021.

[2] K. Al-Hawaj, O. Afuye, S. Agwa, A. Apsel, and C. Batten, "Towards a reconfigurable bit-serial/bit-parallel vector accelerator using in-situ processing-in-sram," in *2020 IEEE International Symposium on Circuits and Systems (ISCAS)*. IEEE, 2020, pp. 1–5.

[3] M. F. Ali, A. Jaiswal, and K. Roy, "In-memory low-cost bit-serial addition using commodity dram technology," *Transactions on Circuits and Systems I: Regular Papers*, vol. 67, no. 1, pp. 155–165, 2019.

[4] T. Andrulis, J. S. Emer, and V. Sze, "Raella: Reforming the arithmetic for efficient, low-resolution, and low-loss analog pim: No retraining required!" in *Proceedings of the 50th Annual International Symposium on Computer Architecture*, 2023, pp. 1–16.

[5] S. Angizi and D. Fan, "Redram: A reconfigurable processing-in-dram platform for accelerating bulk bit-wise operations," in *International Conference on Computer-Aided Design*. IEEE, 2019, pp. 1–8.

[6] A. Boroumand, S. Ghose, Y. Kim, R. Ausavarungnirun, E. Shiu, R. Thakur, D. Kim, A. Kuusela, A. Knies, P. Ranganathan, and O. Mutlu, "Google workloads for consumer devices: Mitigating data movement bottlenecks," in *Proceedings of the Twenty-Third International Conference on Architectural Support for Programming Languages and Operating Systems*, 2018, pp. 316–331.

[7] K. K. Chang, A. G. Yağlıkçı, S. Ghose, A. Agrawal, N. Chatterjee, A. Kashyap, D. Lee, M. O'Connor, H. Hassan, and O. Mutlu, "Understanding reduced-voltage operation in modern dram devices: Experimental characterization, analysis, and mechanisms," *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, vol. 1, no. 1, pp. 1–42, 2017.

[8] Q. Deng, Y. Zhang, M. Zhang, and J. Yang, "Lacc: Exploiting lookup table-based fast and accurate vector multiplication in dram-based cnn accelerator," in *Proceedings of the 56th Annual Design Automation Conference 2019*, 2019, pp. 1–6.

[9] J. D. Ferreira, G. Falcao, J. Gómez-Luna, M. Alser, L. Orosa, M. Sadrosadati, J. S. Kim, G. F. Oliveira, T. Shahroodi, A. Nori, and O. Mutlu, "pluto: Enabling massively parallel computation in dram via lookup tables," in *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2022, pp. 900–919.

[10] F. Gao, G. Tziantzioulis, and D. Wentzlaff, "Computedram: In-memory compute using off-the-shelf drams," in *Proceedings of the 52nd annual international symposium on microarchitecture*, 2019, pp. 100–113.

[11] J. C. Gealow, "Impact of processing technology on dram sense amplifier design," Ph.D. dissertation, Massachusetts Institute of Technology, 1990.

[12] N. M. Ghiasi, N. Vijaykumar, G. F. Oliveira, L. Orosa, I. Fernandez, M. Sadrosadati, K. Kanellopoulos, N. Hajinazar, J. G. Luna, and O. Mutlu, "Alp: Alleviating cpu-memory data movement overheads in memory-centric systems," *IEEE Transactions on Emerging Topics in Computing*, 2022.

[13] N. Hajinazar ,*et al.*, "Simdram: a framework for bit-serial simd processing using dram," in *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2021, pp. 329–345.

[14] K. Itoh, *VLSI memory chip design*. Springer Science & Business Media, 2013, vol. 5.

[15] L. Jiang, M. Kim, W. Wen, and D. Wang, "Xnor-pop: A processing-in-memory architecture for binary convolutional neural networks in wide-io2 drams," in *2017 IEEE/ACM International Symposium on Low Power Electronics and Design (ISLPED)*. IEEE, 2017, pp. 1–6.

[16] B. Keeth, R. J. Baker, B. Johnson, and F. Lin, *DRAM circuit design: fundamental and high-speed topics*. John Wiley & Sons, 2007, vol. 13.

[17] S. Li, D. Niu, K. T. Malladi, H. Zheng, B. Brennan, and Y. Xie, "Drisa: A dram-based reconfigurable in-situ accelerator," in *Proceedings of the 50th Annual IEEE/ACM International Symposium on Microarchitecture*, 2017, pp. 288–301.

[18] S.-L. Lu, Y.-C. Lin, and C.-L. Yang, "Improving dram latency with dynamic asymmetric subarray," in *Proceedings of the 48th International Symposium on Microarchitecture*, 2015, pp. 255–266.

[19] H. Luo, T. Shahroodi, H. Hassan, M. Patel, A. G. Yağlıkçı, L. Orosa, J. Park, and O. Mutlu, "Clr-dram: A low-cost dram architecture enabling dynamic capacity-latency trade-off," in *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2020, pp. 666–679.

[20] M. Marazzi, F. Solt, P. Jattke, K. Takashi, and K. Razavi, "Rega: Scalable rowhammer mitigation with refresh-generating activations," in *44rd IEEE Symposium on Security and Privacy (SP 2023)*. IEEE, 2023.

[21] O. Mutlu, S. Ghose, J. Gómez-Luna, and R. Ausavarungnirun, "Enabling practical processing in and near memory for data-intensive computing," in *Proceedings of the 56th Annual Design Automation Conference 2019*, 2019, pp. 1–4.

[22] A. Olgun, J. G. Luna, K. Kanellopoulos, B. Salami, H. Hassan, O. Ergin, and O. Mutlu, "Pidram: A holistic end-to-end fpga-based framework for processing-in-dram," *ACM Transactions on Architecture and Code Optimization*, vol. 20, no. 1, pp. 1–31, 2022.

[23] G. F. Oliveira, J. Gómez-Luna, S. Ghose, A. Boroumand, and O. Mutlu, "Accelerating neural network inference with processing-in-dram: From the edge to the cloud," *IEEE Micro*, vol. 42, no. 6, pp. 25–38, 2022.

[24] G. F. Oliveira, A. Olgun, A. G. Yağlıkçı, F. N. Bostancı, J. Gómez-Luna, S. Ghose, and O. Mutlu, "Mimdram: An end-to-end processing-using-dram system for high-throughput, energy-efficient and programmer-transparent multiple-instruction multiple-data computing," in *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2024, pp. 186–203.

[25] L. Orosa, Y. Wang, M. Sadrosadati, J. S. Kim, M. Patel, I. Puddu, H. Luo, K. Razavi, J. Gómez-Luna, H. Hassan, N. Mansouri-Ghiasi, S. Ghose, and O. Mutlu, "Codic: A low-cost substrate for enabling custom in-dram functionalities and optimizations," in *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*, 2021, pp. 484–497.

[26] J. Park, R. Azizi, G. F. Oliveira, M. Sadrosadati, R. Nadig, D. Novo, J. Gómez-Luna, M. Kim, and O. Mutlu, "Flash-cosmos: In-flash bulk bitwise operations using inherent computation capability of nand flash memory," in *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2022, pp. 937–955.

[27] N. Rohbani, S. Darabi, and H. Sarbazi-Azad, "Pf-dram: a precharge-free dram structure," in *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2021, pp. 126–138.

[28] N. Rohbani, M. A. Soleimani, and H. Sarbazi-Azad, "Pipf-dram: Processing in precharge-free dram," in *Proceedings of the 59th ACM/IEEE Design Automation Conference*, ser. DAC '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 1075–1080.

[29] S. Roy, M. Ali, and A. Raghunathan, "Pim-dram: Accelerating machine learning workloads using processing in commodity dram," *IEEE Journal on Emerging and Selected Topics in Circuits and Systems*, vol. 11, no. 4, pp. 701–710, 2021.

[30] V. Seshadri, Y. Kim, C. Fallin, D. Lee, R. Ausavarungnirun, G. Pekhimenko, Y. Luo, O. Mutlu, P. B. Gibbons, M. A. Kozuch, and T. C. Mowry, "Rowclone: Fast and energy-efficient in-dram bulk data copy and initialization," in *Proceedings of the 46th Annual IEEE/ACM International Symposium on Microarchitecture*, 2013, pp. 185–197.

[31] V. Seshadri, D. Lee, T. Mullins, H. Hassan, A. Boroumand, J. Kim, M. A. Kozuch, O. Mutlu, P. B. Gibbons, and T. C. Mowry, "Ambit: In-memory accelerator for bulk bitwise operations using commodity dram technology," in *Proceedings of the 50th Annual IEEE/ACM International Symposium on Microarchitecture*, 2017, pp. 273–287.

[32] N. H. Weste and D. Harris, *CMOS VLSI design: a circuits and systems perspective*. Pearson Education India, 2015.

[33] X. Xin, Y. Zhang, and J. Yang, "Roc: Dram-based processing with reduced operation cycles," in *Proceedings of the 56th Annual Design Automation Conference 2019*, 2019, pp. 1–6.

[34] ——, "Elp2im: Efficient and low power bitwise operation processing in dram," in *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2020, pp. 303–314.

[35] R. Zhou, A. Roohi, D. Misra, and S. Angizi, "Flexidram: A flexible in-dram framework to enable parallel general-purpose computation," in *Proceedings of the ACM/IEEE International Symposium on Low Power Electronics and Design*, 2022, pp. 1–6.

[36] ——, "Red-lut: Reconfigurable in-dram luts enabling massive parallel computation," in *Proceedings of the 41st IEEE/ACM International Conference on Computer-Aided Design*, ser. ICCAD '22. New York, NY, USA: Association for Computing Machinery, 2022.

[37] R. Zhou, S. Tabrizchi, A. Roohi, and S. Angizi, "Lt-pim: An lut-based processing-in-dram architecture with rowhammer self-tracking," *IEEE Computer Architecture Letters*, vol. 21, no. 2, pp. 141–144, 2022.